A Project Report

on

Snake Game

Submitted in partial fulfillment of the requirement of

PROJECT-I

(BIT105CO)

BACHELOR OF INFORMATION TECHNOLOGY

Submitted to

Purbanchal University

Biratnagar, Nepal

Submitted By

Dikshant Khanal (311918)

Nishant Shahi (311928)

Rishi Raj Koirala (311934)

KANTIPUR CITY COLLEGE

Putalisadak, Kathmandu

June, 2025

A Project Report

on

Snake Game

Submitted in partial fulfillment of the requirement of

PROJECT-I

(BIT105CO)

BACHELOR OF INFORMATION TECHNOLOGY

Submitted To

Purbanchal University

Biratnagar, Nepal

Submitted By

Dikshant Khanal (311918)

Nishant Shahi (311928)

Rishi Raj Koirala (311934)

Project Supervisor

Er. Ronak Singh Karki

KANTIPUR CITY COLLEGE

Putalisadak, Kathmandu

June, 2025

ABSTRACT

This project presents a console-based Snake Game developed using the C programming language for the Windows platform. The game challenges players to control a snake within a bordered grid, collecting fruits to increase its length while avoiding collisions with walls and its own tail. It includes features such as real-time keyboard input handling, adjustable difficulty levels, a scoring system, and high score tracking for enhanced engagement. Players are given a replay option at the end of each session, encouraging repeated gameplay. The interface is built using ASCII characters and Windows console functions for smooth screen updates without relying on external graphics libraries. The code is modular and structured for easy understanding and future expansion. Arrays are used to manage the snake's body and ensure precise movement and collision detection. This project demonstrates how fundamental C programming concepts can be applied to create interactive games, with future plans to enhance gameplay through the integration of graphical elements.

ACKNOWLEDGEMENTS

We are thankful for the opportunity to present our college project, "Snake Game," and would like to express our appreciation to everyone who supported us throughout its development.

We are especially grateful to our supervisor, Er. Ronak Singh Karki, for his guidance, encouragement, and consistent support throughout the project. His insightful suggestions and feedback were crucial to our success.

This project was the result of a collaborative effort among team members Dikshant Khanal, Nishant Shahi, and Rishi Raj Koirala. Each of us contributed equally to the design, coding, testing, and documentation phases. Our teamwork and commitment were key to accomplishing our objectives.

We also extend our sincere thanks to our college and faculty for providing the resources and environment necessary for completing this project.

Sincerely,

Dikshant Khanal

Nishant Shahi

Rishi Raj Koirala

PREFACE

This project report titled "Snake Game" is the result of our effort to develop a simple yet engaging console-based game using the C programming language. The primary goal of this project was to apply our theoretical knowledge of programming in a practical and interactive way.

Throughout this project, we focused on implementing fundamental game mechanics such as real-time user input, collision detection, score tracking, difficulty levels, and high score management — all within a text-based console interface. Despite its simplicity, the project challenged us to think logically and solve problems through structured coding and teamwork.

The development of this game has strengthened our understanding of the C language and enhanced our collaboration and problem-solving skills. We hope this report provides helpful insights for students or beginners who are interested in developing simple games and learning the foundational concepts of software development.

We would like to express our sincere gratitude to our supervisor, Er. Ronak Singh Karki, for his valuable guidance, support, and encouragement throughout the project.

DECLERATION

We declare that this project report titled "Snake Game", submitted in partial fulfilment of the Bachelor's program, is a record of original work carried out by us under the supervision of Er. Ronak Singh Karki and has not formed the basis for the award of any other degree or diploma, in this or any other Institution or University. In keeping with the ethical practice in reporting the scientific information, due acknowledgements have been made wherever the findings of others have been cited.

Dikshant Khanal
Nishant Shahi
Rishi Raj Koirala

SUPERVISOR'S APPROVAL

This is to certify that the major project entitled Snake Game in C undertaken and demonstrated by Dikshant Khanal, Nishant Shahi, Rishi Raj Koirala has been successfully completed under my supervision as a partial fulfillment of the requirements for the degree of BIT, 1st Semester under Purbanchal University. I, henceforth, approve this project to be awarded the certificate by the concerned authority.

During supervision, I found students hardworking, skilled and ready to undertake any professional work related to this field in future.

_____

Er. Ronak Singh Karki
Project Supervisor
Department of IT

CERTIFICATE FROM DEPARTMENT

Following the Supervisor's Approval and Examiners' Acceptance, the project entitled "Snake Game" submitted by Dikshant Khanal, Nishant Shahi, and Rishi Raj Koirala as a partial fulfillment of the requirements for the degree of BIT, 1st Semester under Purbanchal University, has been officially awarded by this certificate.

I wish the students all the best for their future endeavours.

Saroj Pandey	Rubim Shrestha
HoD	Project Coordinator
Department of IT	Department of IT

TABLE OF CONTENT

ABSTRACT iii

ACKNOWLEDGEMENTS iv

PREFACE v

DECLERATION vi

SUPERVISOR'S APPROVAL vii

1. Introduction 1

1.1 Overview 1

1.2 Problem Statement 1

1.3 Objectives 1

1.4 Features 2

1.6 Scope and Limitations 2

2. Literature Review 3

2.1 System 1 Overview – Snake Game (C Language) 3

2.1.1 System 1 Pros 3

2.1.2 System 1 Cons 3

2.2 System 2 Overview – Snake Game (C Language) 4

2.2.1 System 2 Pros 4

2.1.2 System 1 Cons 4

3. Methodology 5

3.1 Software Development Life Cycle 5

3.2 Technologies and Tools Used 6

3.2.1 Programming Language Used 6

3.2.2 Database Used 7

3.2.3 Software Used 7

3.2.4 Integrated Development Environment (IDE) 7

3.3 Assignment of Roles and Responsibility 7
4. System Analysis 8
4.1 Requirements Analysis 8
4.1.1 Requirement Gathering 8
4.1.2 Functional Requirements 9
4.2 Gantt Chart 10
5. System Design 11
5.1 System flow Diagram and Algorithm 11
5.1.1 System Flow 11
5.1.2. System Diagram 12
5.1.3 System Flow(collision) 13
5.1.4. System Diagram(collision) 14
6. System Development and Implementation 15
6.1 Programming Platform 15
6.2 Operating Environment 15
7. Testing and Debugging 16
7.1 Tools Used in Testing 16
7.2 Test Cases 17
8. Conclusion 18
9. References 19

LIST OF FIGURES

Figure 3.1: Waterfall Model……5

Figure 4.2 Gantt chart…….…….……10

Fig 5.1.2 System Flow Diagram…….....12

Fig 5.1.2 System Flow Diagram(collision)…….....14

1. Introduction

1.1 Overview

The Snake Game is a timeless arcade classic where players control a snake navigating a grid, consuming food to grow longer while avoiding collisions with the walls and itself. This C-based implementation offers a console-driven experience, showcasing key programming concepts like real-time updates, collision detection, and user input management. It serves as both an educational tool and a source of entertainment, especially for beginner programmers looking to strengthen their understanding of the C language.

1.2 Problem Statement

In today's fast-paced and stressful world, people often seek simple and engaging games as a way to relax and pass time. However, many modern games demand high-end graphics and advanced hardware, making them less accessible to users with basic devices. There is a need for a lightweight, entertaining game that is easy to play, requires minimal system resources, and still offers a challenge. The Snake Game addresses this by providing an addictive, fast-paced gameplay experience in a simple console environment. It offers casual players a fun way to test their reflexes and decision-making without requiring any expensive gaming setup.

1.3 Objectives

- To develop a lightweight and accessible console-based Snake Game
- To provide an engaging and challenging gameplay experience

1.4 Features

- **Welcome page:** A welcome page to welcome the player
- **Scoring System:** Tracks and displays the player's score during gameplay.
- **Difficulty Levels:** Allow players to select game speed (Easy, Medium, Hard).
- **High Score Saving:** Stores and updates the best score of each levels using a file.
- **Restart:** Allows the player to restart the game

1.5 Significance

This project highlights how simple programming techniques can be utilized to build a functional and engaging game. It is particularly useful for beginners learning C programming, as it integrates key concepts such as real-time input, arrays, loops, and conditional logic. The Snake Game also offers an enjoyable form of entertainment that is accessible even on low-specification systems, making it suitable for users with limited hardware resources.

1.6 Scope and Limitations

The scope of this project is confined to a text-based, console game developed in the C programming language. It does not feature advanced graphics, sound effects, or multiplayer functionality. The game is designed to run on systems that support C console applications and may require modifications for compatibility with different operating systems. Additionally, the interface is limited to basic text rendering, which may restrict the visual appeal compared to modern graphical games.

1.7 Organization of the Document

This document is divided into chapters to give a clear understanding of the Snake Game project. It starts with an introduction covering background, objectives, features, and scope. The Literature Review analyzes similar games and their design. Methodology outlines the development process, while System Design explains the architecture and core logic. A Work Schedule presents the timeline, and the document ends with references and appendices for additional insights.

2. Literature Review

To assess the relevance and impact of a console-based game like the Snake Game, existing simple arcade-style games implemented in basic programming languages were reviewed. The focus was on games that emphasize core programming concepts, such as real-time input handling, collision detection, and dynamic game state management without relying on advanced graphics. This review highlights the strengths and limitations of similar projects, such as their educational value and accessibility, as well as gaps like lack of scalability or limited features. The Snake Game project aims to address these gaps by providing a well-rounded, engaging, and easily portable game that reinforces foundational programming skills while offering an enjoyable user experience.

2.1 System 1 Overview – Snake Game (C Language)

The Snake Game is a classic arcade game that has been widely implemented across various programming languages, including C. This review focuses on a C-based implementation using libraries such as conio.h and windows.h for screen control and timing. The game mechanics are simple yet engaging: the snake moves continuously, consumes food to grow, and the game ends if it collides with walls or itself. The program is organized into modules like setup (initializing variables), rendering (displaying elements), input handling (real-time control), and game logic (movement, collisions, scoring). This clear structure helps demonstrate fundamental programming concepts while offering enjoyable gameplay.

2.1.1 System 1 Pros

- Simple and efficient C implementation.
- Real-time user interaction.
- Effective use of arrays for movement tracking**.**

2.1.2 System 1 Cons

- Basic graphics with no enhancements.
- No difficulty levels or extra mechanics.
- Potential screen flickering due to system("cls").

2.2 System 2 Overview – Snake Game (C Language)

The Snake Game is a classic arcade-style game where the player controls a snake navigating within a confined area. The objective is to consume randomly appearing food items, which causes the snake to grow longer with each meal. The game challenges players to skill fully control the snake using the keyboard arrow keys, avoiding collisions with the boundaries or the snake's own body. As the snake consumes more food, the game becomes progressively faster and more challenging, enhancing the excitement and engagement. The game features a score-tracking system that records the number of food items eaten, providing a simple yet addictive gameplay experience that tests reflexes and strategic movement.

2.2.1 System 2 Pros

- Simple Gameplay: Easy to learn and play for all ages
- Low Development Cost: Minimal resources needed for development
- Engaging: Addictive gameplay focused on score improvement

2.1.2 System 1 Cons

- Limited Complexity: Gameplay can become repetitive, reducing long-term interest.
- No Advanced Features: Lacks modern elements like multiplayer or power-ups.
- Not Visually Rich: Simple design may not attract players seeking immersive graphics.

3. Methodology

The development of the Snake Game followed a structured approach based on the Software Development Life Cycle (SDLC) to ensure a systematic and efficient process. Starting with requirement analysis and design, the game was implemented as a console-based application in C using standard and Windows-specific libraries for input handling and screen management. Key features such as snake movement, food generation, collision detection, score tracking, and dynamic speed adjustment were modularly developed for clarity and maintainability. Additionally, file handling was incorporated to save and load the high score, enhancing user engagement. The game underwent iterative testing to ensure stability and responsiveness, resulting in an engaging and functional terminal-based game.

3.1 Software Development Life Cycle

The Snake Game was developed following the Waterfall Model of the SDLC, ensuring a clear and sequential development process.

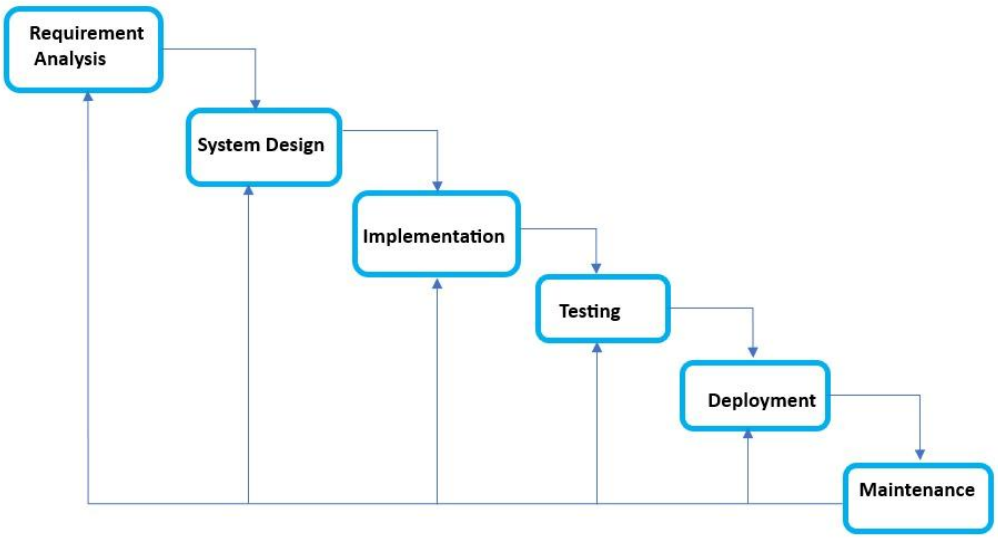

Figure 3.1 Waterfall Model

- **Requirement Analysis:** Defined core gameplay elements such as snake movement, food placement, scoring, speed variation, collision detection, and user controls.

- **System Design:** Created the overall game architecture, including game loop flow, screen rendering, input handling, tail growth mechanics, and file handling for high score persistence.

- **Implementation:** Developed the game using C programming language, structuring the code into modular functions for better organization and maintainability.

- **Testing:** Performed functional testing and iterative manual gameplay to identify and fix bugs, balance game difficulty, and optimize user experience.

- **Deployment & Maintenance:** Delivered a stable console-based game executable, with ongoing improvements based on user feedback and performance observations.

3.2 Technologies and Tools Used

3.2.1 Programming Language Used

C Language:

The project was developed using the C programming language due to its efficiency, simplicity, and low-level access to memory and hardware features. It is especially well-suited for creating terminal-based games, allowing real-time keyboard input and control over console output.

3.2.2 Database Used

File System:

Instead of using a traditional database, file handling was implemented using C's standard I/O functions to store and retrieve high scores. This approach ensures persistent data across sessions while keeping the game lightweight and easy to manage

3.2.3 Software Used

GCC Compiler

Used for compiling the C source code on Windows, providing compatibility with Windows console APIs.

Command Prompt (CMD)

The Windows terminal was used to run the game and view real-time game output during development and testing.

3.2.4 Integrated Development Environment (IDE)

Dev C++

Dev C++ served as the primary code editor and IDE for the project. It supports C/C++ programming with built-in compilation, syntax highlighting, error detection, and basic debugging features. Its simplicity and integration with the GCC compiler made it suitable for developing console-based games.

3.3 Assignment of Roles and Responsibility

Member Name	Rules and Responsibility
Dikshant Khanal	Coding, Documentation
Nishant Shahi	Coding, Documentation, literature review, gantt chart
Rishi Raj Koirala	Coding, Documentation, Algorithm, Flowchart Literature review

4. System Analysis

This chapter presents the analysis of the Snake Game system, focusing on its objectives, functional components, and performance expectations. The analysis phase played a key role in defining how the system should operate, what features it must support, and the conditions it must fulfill. Emphasis was placed on both functional and non-functional requirements to ensure responsive gameplay, intuitive controls, and minimal system resource consumption. The goal was to design a smooth, engaging experience while maintaining simplicity and efficiency throughout the game's execution.

4.1 Requirements Analysis

System analysis began with identifying the needs of the end user and translating those into precise technical requirements. The development was guided by the constraints of a console-based environment, real-time input handling, and simplicity of controls. Key considerations included ensuring responsive gameplay, a user-friendly interface, and support for features such as score tracking, difficulty levels, and game over conditions.

4.1.1 Requirement Gathering

The requirements were gathered through:

Requirement ID	Description
R1	Studying similar console-based games such as Snake and classic arcade games to understand basic gameplay mechanics.
R2	Understanding the limitations and capabilities of console/terminal environments for smooth rendering and input handling.
R3	Personal goals to implement features like real-time movement, collision detection, score display, and gradual difficulty increase.

4.1.2 Functional Requirements

Functional requirements describe what the system should do. For Snake Game, the functional requirements are:

Function ID	Functional Requirement	Description
F1	Start Game	Player selects "Start Game" from main menu to begin gameplay
F2	View leaderboard	User accesses high score by selecting view leaderboard (option 2) in main menu
F3	Exit application	User quits program by selecting exit (option 3)
F4	Difficulty level	User chooses difficulty (for easy = 1, for medium = 2, for hard = 3)
F5	Control snake movement	User navigates snake by using ‘W’ A S D keys during gameplay
F6	Quit current game section	User ends active gameplay by pressing ‘X’
F7	Track score progress	User see current score and difficulty-specific high score during gameplay
F8	View game over summary	User sees final score and high score comparison after game ends.
F9	Restart after game over	User chooses replay by pressing ‘Y’ after game ends
F10	Return to main menu	User exists leaderboard view by pressing any key and return to main menu

4.2 Gantt Chart

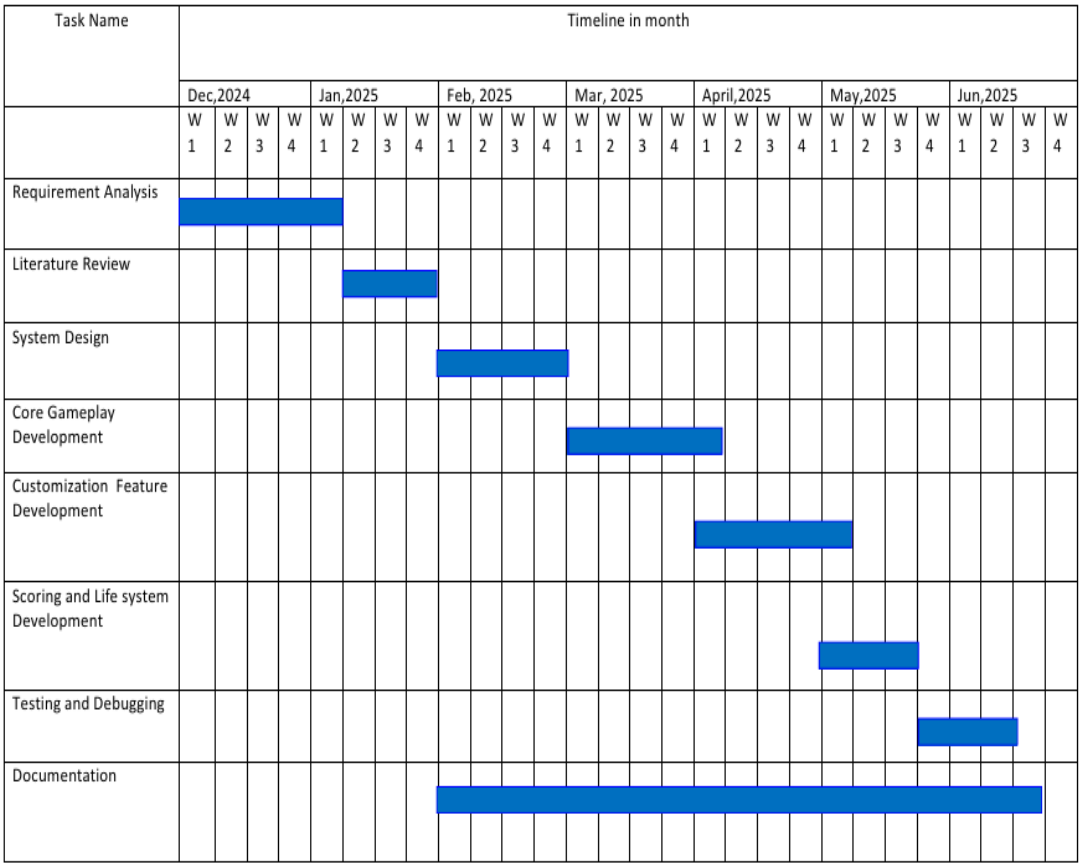

Figure 4.2 Gantt chart

5. System Design

5.1 System flow Diagram and Algorithm

This section explains the system architecture, design, and how the game operates.

5.1.1 System Flow

1. Start game
2. Display welcome screen
3. Show the main menu
    - If the user selects "Start Game", go to Step 4.
    - If the user selects "Leaderboard", display the leaderboard and return to the Main Menu.
4. Show Start Menu
5. Select level(difficulty)
6. Enter Game Loop (if gameover == false)
    - If gameover == false, continue
    - Else, go to Step 14
7. Draw Game Field
8. Get Player Input
9. Update Snake Position
10. Check Collisions
    - If collision detected, proceed to Step 11
    - Else, continue Game Loop
11. If wall collision, then Game Over
12. If self-collision, then Game Over
13. If fruit eaten, then Increase Score & Grow Tail & Adjust Speed & Continue Loop
14. Game Over
15. Ask: "Do you want to play again?
    - If Yes, reset game variables and go to Step 2
    - Else, go to Step 17
16. Display Game Over Message
17. End Program

5.1.2. System Diagram

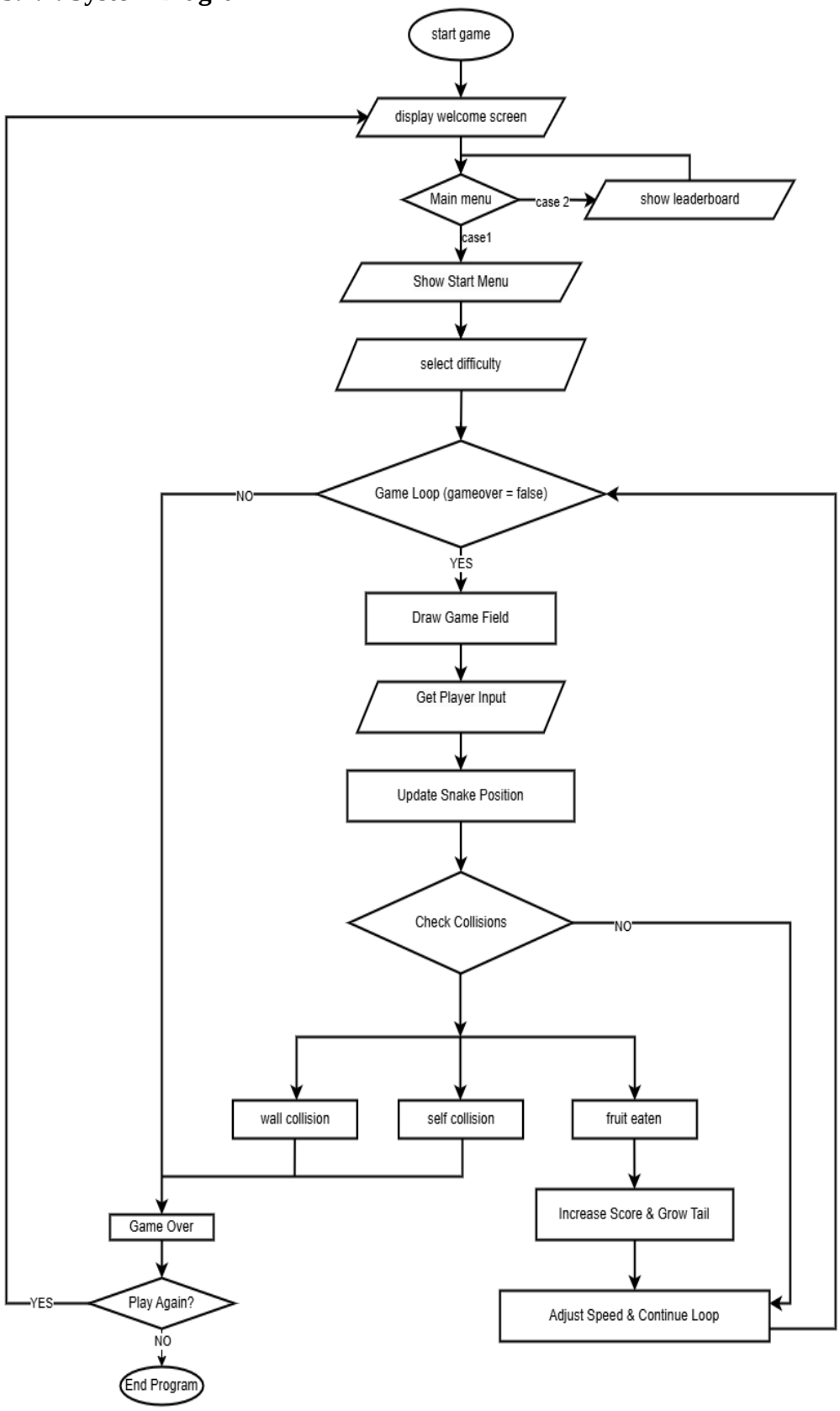

Fig 5.1.2 System Flow Diagram

5.1.3 System Flow(collision)

1. Start
2. Update snake position
3. Check wall collision
    - If snake's head crosses game boundary, then game over
4. Check self-collision
    - If snake's head matches any body segment, then game over
5. Check fruit collision
    - If snake's head matches fruit position
        - Grow snake
        - Generate new fruit
6. Render game
7. Repeat loop until game over
8. Stop

5.1.4. System Diagram(collision)

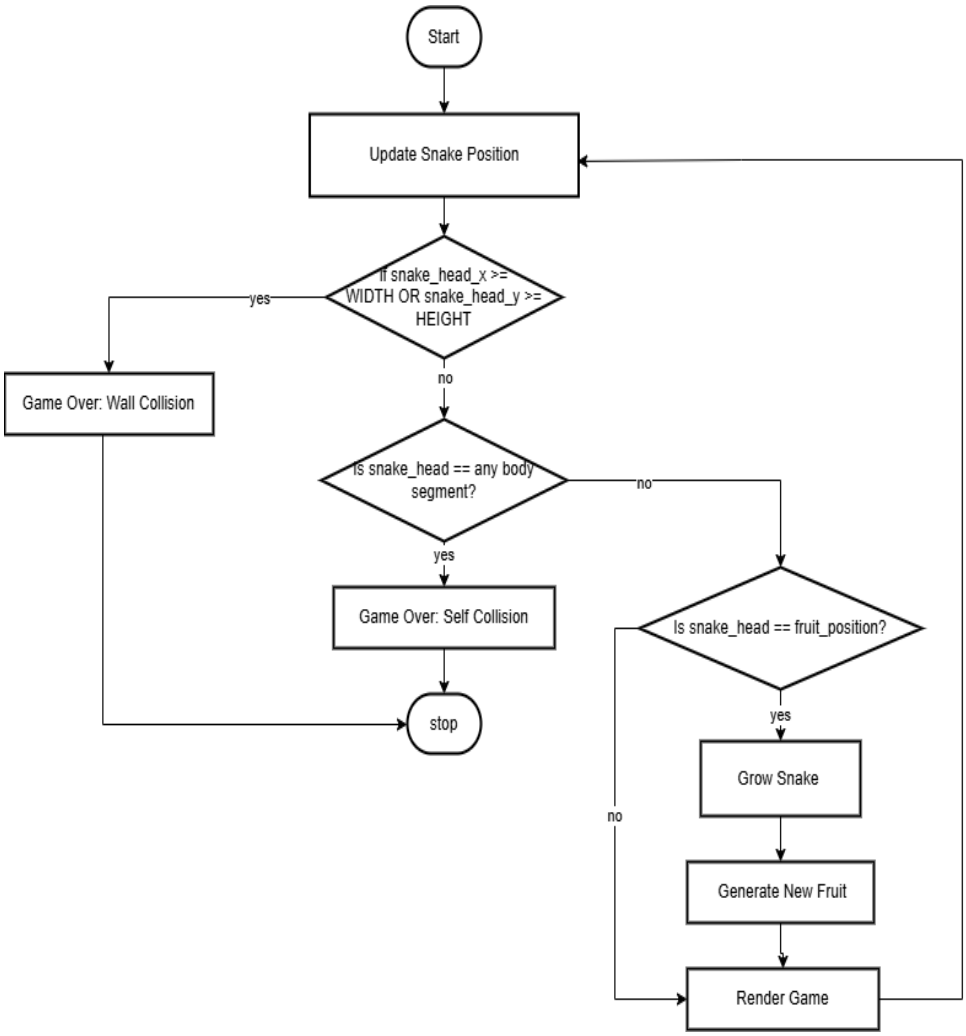

Fig 5.1.4 System Flow Diagram(Collision)

6. System Development and Implementation

This chapter details the development process and the implementation environment of the Snake Game. It includes the selected programming platform, necessary setup for smooth execution, and the tools and technologies used during development.

6.1 Programming Platform

The Snake Game was developed using the C programming language for its simplicity, efficiency, and control over system resources. The project was built as a console-based application, enabling real-time user input and efficient screen rendering using basic functions.

6.2 Operating Environment

The game was developed and tested in a Windows environment using Dev C++. It runs in the Windows command prompt, utilizing conio.h and windows.h libraries for input handling, cursor control, and timing functions. This setup ensures ease of use and requires no external graphical libraries.

7. Testing and Debugging

This chapter focuses on the strategies, tools, and methods applied to test and debug the Car Dodging Game, ensuring the program functions correctly and efficiently.

7.1 Tools Used in Testing

S.N	Tool	Specifications/Details
1	Hardware	Laptop with Intel i5 processor, 8GB RAM
2	Development Environment	Dev C++ (Windows) with built-in compiler and debugger
3	Manual Debugging	Step-by-step code tracing, print statements for logic verification
4	Manual Test	Console-based gameplay testing to check controls, collision, and scoring

7.2 Test Cases

ID	Test Case Name	Test Case Description	Test Case Data	Expected Result	Actual Result	Status
1	Game Lunch	Start the Game from terminal	Run executable	Game start and displays intro screen	Game starts and displays intro	Success
2	Player Movement	Move snake using arrow keys	Arrow key input	Snake moves in the input direction	Snake moves correctly	Success
3	Collision Detection	Detect collision with wall or body	Snake hits wall/body	Game over triggered	Game over triggered	Success
4	Score Increment	Successfully eat food	Snake eats food	Score increases by 10 points	Score increases by 10 points	Success
5	Snake Growth	Snake should grow after eating food	Player overlaps with heart	Snake eats food	Snake length increases	Success
6	Game Over Condition	Trigger game over on collision	Snake collides	“Game Over” message shown	“Game Over” message shown	Success
7	High Score Saving	Save high score between sessions	Play game, quit, restart	High score is saved and displayed	High score saved and displayed	Success

8. Conclusion

The Snake Game project successfully demonstrates the application of fundamental programming concepts in C within a console-based environment. Through effective use of logic, control structures, and file handling, a functional and engaging game was developed. The system supports smooth gameplay, real-time user interaction, and essential features such as score tracking and collision detection. The project also reflects careful planning, modular design, and rigorous testing, ensuring both functionality and reliability. Overall, this project highlights how classic games can be recreated using simple tools and programming techniques, reinforcing core coding skills and game development principles.

9. References

[1] GeeksforGeeks, “C Programming Language,” https:// www.geeksforgeeks.org /. [Accessed: 15-Nov-2024]. [Online]. Available:

[2] GitHub, “Snake Game in C,” [Online]. Available: https://github.com. [Accessed: 18-Nov-2024].